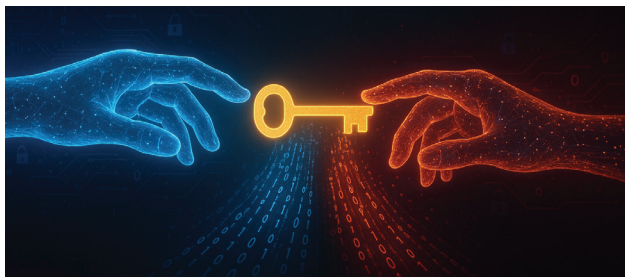


SageMath: Implementing Secure Key Exchange

Discover why public-key cryptography is a powerful mechanism for secure key exchange in this 12th part of the series of articles on SageMath.



In the previous article in this series (published in the September 2025 issue of *Open Source For You*, we explored RSA in detail and implemented a simplified yet practically secure version of it. Unlike the toy examples we had considered earlier, this implementation can be executed even on moderately powered computers, yet remains unbreakable with the resources of the most powerful supercomputers available today. However, we also observed that RSA, like most public-key cryptographic (PKC) systems, is computationally intensive and therefore not well suited for large-scale data encryption. In practice, PKC is typically employed to encrypt short messages or, more importantly, to securely exchange secret keys between two parties. Once exchanged, these keys are then used in symmetric-key cryptography, which is significantly faster and better suited for encrypting large volumes of data. This combination makes public-key cryptography a powerful mechanism for secure key exchange—an area that will be the central focus of this article.

Before moving further, let us revisit another thread we began developing in the previous article—why SageMath is faster and more efficient than Python? In that discussion, we examined examples that highlighted the syntactical differences between SageMath and Python, demonstrating that SageMath is not merely another dialect of Python. We also observed cases where SageMath provided significant speedups while performing operations similar to those in Python.

We concluded that this performance advantage arises because SageMath leverages specialised mathematical libraries such as FLINT (Fast Library for Number Theory) and GMP (GNU Multiple Precision Arithmetic Library). As promised, in this article we will continue exploring this topic by discussing additional specialised libraries and back-end

tools that empower SageMath to deliver superior performance compared to other languages commonly used in scientific computing, such as Python and Julia.

Let us now take a moment to introduce Julia. Julia is a modern, general-purpose programming language released in 2012, developed with a strong focus on scientific and numerical computing. In one of my earlier articles for *OSFY*, published as part of a series on AI, I compared the performance of Julia with Python. The article is available online on the *OSFY* portal at <https://www.opensourceforu.com/2023/07/ai-beyond-python>. If you read it, you will see that Julia clearly outperforms Python when performing similar tasks. Now, let us set up a competition between SageMath and Julia to see who wins.

Factoring a large integer serves as a good test case, given its direct relevance to our ongoing discussion of public-key cryptography (PKC). The Julia program named ‘*juliatime35.jl*’ shown below measures the time taken to compute the factors of a very large integer stored in a variable named *n*.

```
using Primes
n = big"152415787806736785461057782810547205156226"
start = time( )
factors = factor(n)
finish = time( )
println("Time taken: $(finish - start) seconds")
factor_list = collect(keys(factors))
println("The factors are $factor_list")
```

The SageMath program named ‘*sagetime36.sage*’, shown below, performs the same operations as the Julia program ‘*juliatime35.jl*’.

```
n = 152415787806736785461057782810547205156226
t0 = walltime( )
factors = factor(n)
print("Wall time:", walltime(t0), "seconds")
print("The factors are:", factors)
```

Upon executing the programs ‘*juliatime35.jl*’ and ‘*sagetime36.sage*’, the outputs shown in Figure 1 are obtained. The output clearly shows that in the speed ‘competition’,